

Performance Comparison of SGD, XGBoost, and Deep Neural Networks on the dataset of the Higgs Boson Machine Learning Challenge

Software and Computing for Nuclear and Subnuclear Physics
Daniele Bonacorsi, Alessandra Fanfani, Tommaso Diotallevi

Federica Naldi

July 2026

1 Introduction

1.1 The Higgs Boson Machine Learning Challenge

The Higgs Boson Machine Learning Challenge was organized by the ATLAS experiment, which provided a simulated dataset used by the collaboration to search for the Higgs Boson. The training dataset contains background events and signal events, which simulate the production of the Higgs Boson. The purpose of the competition was to promote the collaboration among physicists and data scientists, as machine learning is an extremely powerful tool to analyze the data that is collected by detectors at the LHC. In particular, the aforementioned dataset was meant to be used to train binary classifiers that could correctly discriminate Higgs Boson events from background events. The competition was hosted by Kaggle, and a prize of 7000\$ was promised to the developer of the best binary classifier.

1.2 Project Purposes

This project aims at comparing the performances of three different architectural approaches: the linear SGD classifier, the decision-tree-based XGBoost classifier and a Keras/TensorFlow Deep Neural Network evaluated on the training dataset of the Higgs Boson Machine Learning Challenge. The original training set was inspected, subjected to exploratory data analysis and

pre-processed; then, it was split into the actual training set and the test set. The estimators were evaluated using the following metrics: accuracy, precision, recall, f1-score and ROC-AUC score. On the other hand, the *approximate median significance* is the official metric of the competition and takes into account the statistical weights of events. However, for the purpose of this project it could be neglected, and the ROC-AUC score was used to compare the different performances. In particular, the XGBoost classifier produced the best overall performance when the ROC-AUC score (0.90) and the execution time during the training phase (9.9 s) are considered simultaneously. After that, the impact on XGBoost performance of three possible improvements was investigated; however, while they affected the training time of the model, none of those methods alone influenced the ROC-AUC score significantly.

2 Data Structure and Data Inspection

The initial dataset has a tabular structure and is made of 250000 rows (observations) and 33 columns, which include the event identifier, primary and derived kinematical features, statistical weights and labels.

2.1 Dataset Description

Below, the content of the 33 columns is listed:

- **EventId**: numerical identifier associated to any collision event. It was excluded from the training set as it lacks predictive power.
- **Kinematical Features (30 variables)**: among these, *PRI_jet_num* is discrete (type int64), while the other 29 are continuous (type float64). They are divided into:
 - *Primary (PRI_)*: physical quantities directly measured by subdetectors;
 - *Derived (DER_)*: complex kinematical quantities derived from primary variables.

Some entries contain the "-999.0" value, which indicates that the actual value for that event could not be computed or was meaningless. The kinematical features were saved in **X_data** DataFrame, which has the shape (250000, 30): 250000 horizontal vectors with 30 entries each.

- **Weight:** Statistical weight of the event; it enters the definition of the official evaluation metric of this problem, the *Approximate Median Significance* (AMS). Since this project did not use this metric, weights were excluded from the dataset;
- **Labels:** the original label, expressed as a string variable ("s" for signal and "b" for background). They were saved in `y_data` DataFrame, which has the shape (250000,): 250000 horizontal vectors with one entry each.

2.2 Label Class Distribution

The inspection of the dataset showed a moderate imbalance in the label class distribution, described in Table 1:

Label	Number of Events	Percentage
b	164333	$\approx 65.7\%$
s	85667	$\approx 34.3\%$

Table 1: Distribution of target labels of the dataset, with "b" identifying a background event, and "s" identifying a Higgs Boson event.

3 Exploratory Data Analysis

In this part, the distribution of some features was qualitatively analyzed in order to investigate how to handle meaningless values. In output, the distributions were plotted both for all the values of that feature and for background and signal events separately; however, only two examples were shown below in Figure 1 and 2. While the distribution of angle-related features such as *PRI_tau_eta* is symmetric (Figure 1), the distribution of mass-related features such as *DER_mass_MMC* (Figure 2) and momentum-related features is asymmetric. This suggests that that the median needs to be used to fill `np.nan` values, as it is more suitable to handle asymmetric distribution. However, this step is going to be developed in Section 4.

4 Data Preprocessing

The dataset needs to be processed so that it can be handled correctly by the estimators that are going to be trained. While inspecting the dataset, two criticalities emerged:

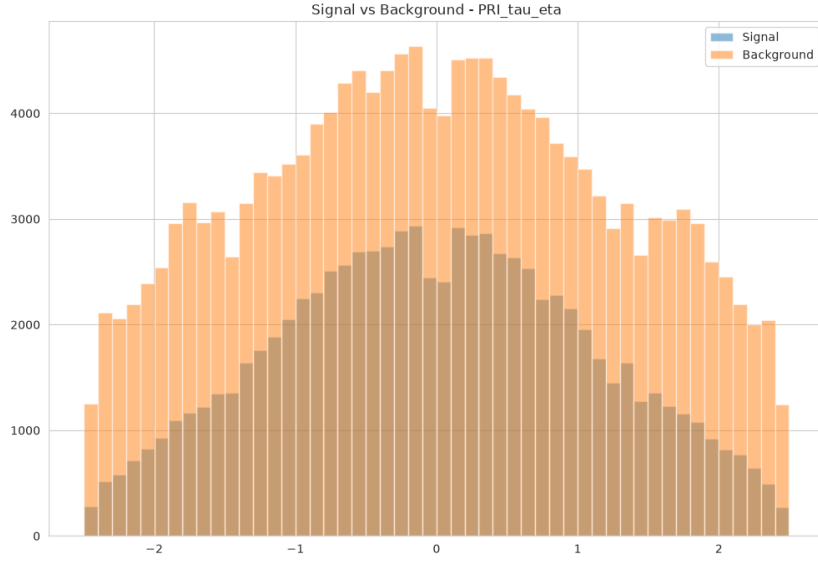


Figure 1: Distribution of PRI_tau_eta for signal (blue) and background (orange) events separately, over the whole initial dataset ($\mathbf{X_data}$). The distribution is manifestly symmetric.

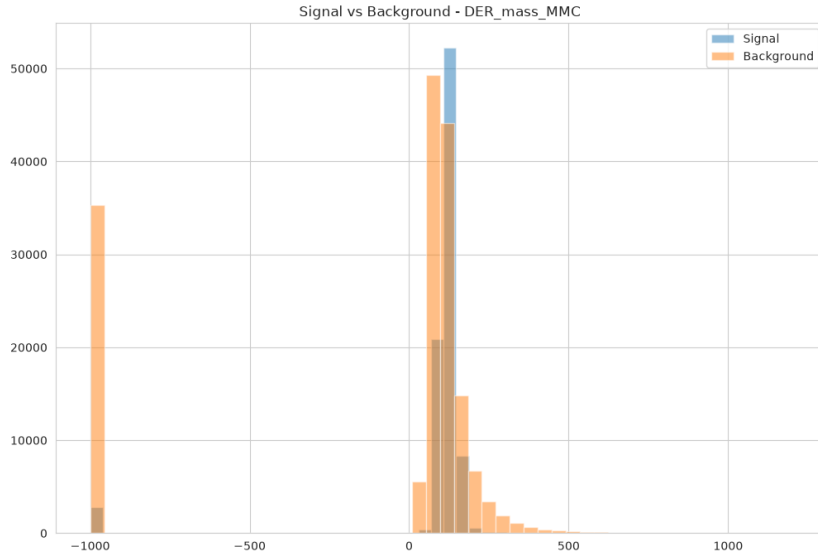


Figure 2: Distribution of DER_mass_MMC for signal (blue) and background (orange) events separately, over the whole initial dataset. The distribution is manifestly asymmetric. Note that the outliers at -999.0 are actually null values, which are going to be handled in Section 4.

- **Labels are expressed as string variables.** Most machine learning models require labels to be integers. Since the problem consists of binary classification, "b" and "s" were replaced with "0" and "1", respectively;
- **Null values are expressed as float variables.** The fact that meaningless - or null - values are written as -999.0 falsifies the statistics and, therefore, the performance of the model. For this reason, they were replaced with the `np.nan` value, which is the standard representation for null values in NumPy. While the XGBoost classifier can handle `np.nan` variables correctly, the SGD classifier and the Keras/TensorFlow DNN require imputation: after splitting the initial dataset into the actual training set (`X_train`, `y_train`) and the test set (`X_test`, `y_test`) with the `train_test_split()` method from the `Scikit-Learn` library (in module `sklearn.model_selection`), the `np.nan` values of each feature were replaced with the median of the corresponding feature. A few comments on this procedure:

- the dataset needs to be split in training set (80%) and test set (20%) before filling `np.nan` values with the median of the feature. If not, the training set would contain statistical information from the test set. Therefore, the training set and the test set would not be completely separated, and the evaluation of the models would be falsified;
- When splitting the dataset, the option "stratify" is switched on (`stratify = y_train`) in order to preserve the label distribution both in the training set and the test set;
- the median was chosen over the mean as the distribution of some features (e. g. *DER_mass_MMC*) is strongly asymmetric - while the mean is biased towards the most likely value, the median is a better representative of the distribution (see Section 3);
- in principle, since the XGBoost classifier can handle `np.nan` values, it could have been trained on a separate training set in which `np.nan` values were not replaced. This option was investigated in the second part of the project (see Section 7); instead, in this part of the project, it seemed more consistent to compare the performance of the different architectures on the same training set.

5 Description and Training of an SGD Classifier, an XGBoost Classifier and a Keras TensorFlow Deep Neural Network

Three models from different algorithmic families were tested: a linear model (SGD), a tree-based model (XGBoost), and a deep neural network model (Keras/TensorFlow DNN). This selection enables a comparison between approaches with different levels of complexity and different abilities to capture relationships within the data.

5.1 Description of the Models

- The SGD classifier implements regularized linear models with stochastic gradient descent; when the classifier was defined, the logistic regression was chosen as the loss function (`Loss = "log_loss"`), as it is the appropriate loss function for binary classification problems.
- the XGBoost Classifier implements machine learning algorithms under the Gradient Boosting framework, based on Decision Trees. When the classifier was defined, AUC was chosen as the evaluation metric for consistency with the metric that is going to be used to compare the models (See Subsection 6.1). For binary classification problems, along with this choice, the "learning objective" must be set to "binary:logistic". Finally, the "tree_method" is chosen to be "hist" to optimize computational efficiency and reduce training time;
- a very basic Deep Neural Network was defined, as the architecture modeling goes beyond the purposes of this project. A sequential model, in which each layer is connected to the next one only, was used, as it is suitable for a simple binary classification problem. Three dense layers were added: the first hidden layer has 64 neurons, the second one has 32 neurons, while the output layer is made of 1 neuron. In particular, the number of neurons was chosen to provide sufficient representational capacity while keeping the model relatively simple and reducing the risk of overfitting. For the first two layers, the non-linear "relu" activation function was used, as non-linearity allows to capture potential non-linear relations between the features of the dataset. On the other hand, for the output layer the "sigmoid" function was chosen, as it is more suitable for binary classification problems since it returns a number between 0 and 1 (a probability). The architecture was displayed in Figure 3. The DNN was compiled with the "Adam" optimizer, which

implements a non-linear algorithm based on the stochastic gradient descent that ensures fast and stable convergence. The "BinaryCrossentropy" loss function was chosen, as it is suitable for binary classification problems. Finally, the accuracy and AUC were chosen as evaluation metrics, with the latter being actually more relevant as it is going to be used to compare the three estimators (see Section 6.1).

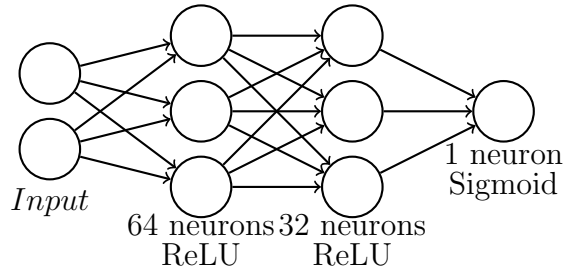


Figure 3: Architecture of the Deep Neural Network used for binary classification. Note that the input layer actually has as many neurons as the number of features.

5.2 Training of the models

While the XGBoost classifier was trained directly on `X_tr` and `y_tr`, the SGD Classifier and the DNN require that each feature looks like a standard normally distributed variable to behave correctly; for this reason, these two models were trained on a dataset scaled with the `StandardScaler()` method from the `Scikit-Learn` library (in `sklearn.preprocessing` module).

5.2.1 Training of the DNN

During the training process of the Keras/TensorFlow DNN, the training set (80% of the initial dataset) is split into the actual training set (80%) and a validation set (20%). The batch size is 128, which means that 128 events are taken at a time and the process is iterated until the full training set is used ($0.8 \times 0.8 \times 250000 / 128 = 1250$ times). The whole process is repeated for 10 epochs. The fit results were plotted to monitor overfitting, and the graphs in Figure 4 were obtained. In this case, the plots show no presence of overfitting, and, in principle, the number of epochs could have been increased to improve the performance of the DNN before it started producing overfitting. However, architecture modeling goes beyond the purposes of this project and, to add to this, these variations would have implied an increase in the execution time

which would have made the DNN less competitive with respect to XGBoost performances.

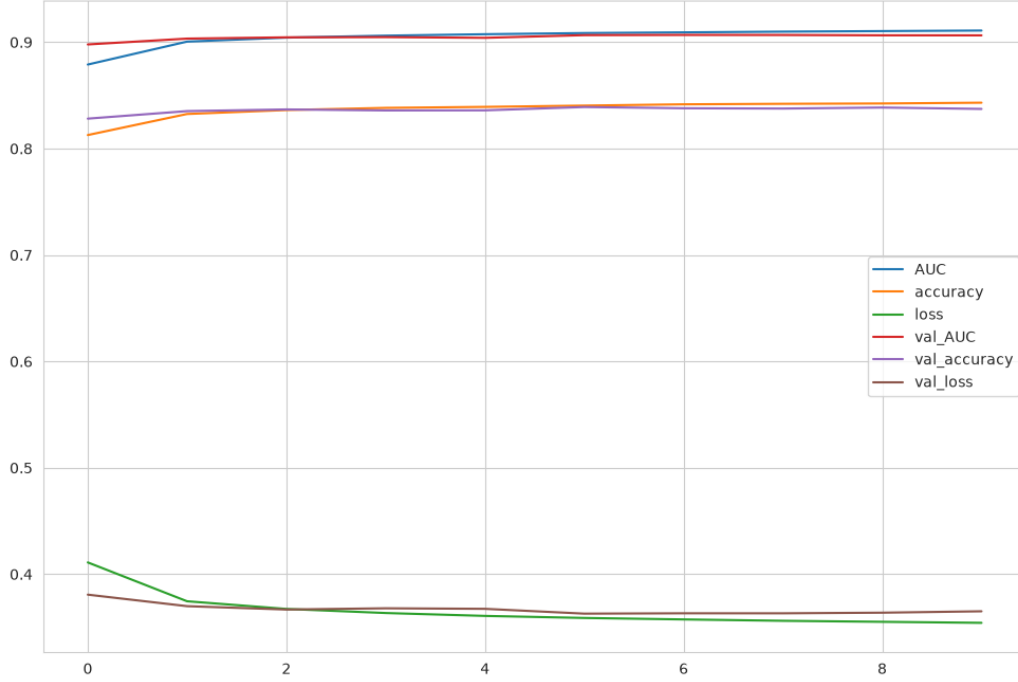


Figure 4: Plot of loss function (green and brown), accuracy (orange and purple) and AUC (blue and red) evaluated on the training set and on the validation set in order to monitor overfitting during the training phase.

6 Evaluation of the Models

The models were evaluated on the test using the following metrics: accuracy, precision, recall, F1-Score and ROC-AUC. The performance of each estimator was evaluated directly on the test set `X_test` for XGBoost or on a scaled (again, with `StandardScaler()`) test set `X_test_scaled` for the DNN and SGD. The evaluation step did not rely on cross validation, as the training set of the Boson Machine Learning Challenge is large enough to make this step negligible. To add to this, this choice was supported by the fact that avoiding cross validation reduced the execution time, which must be taken into consideration as this project is not meant to run on GPU.

6.1 Metrics Description

The models were compared taking into account the following information about the aforementioned metrics:

- The **accuracy** is defined as the ratio of correct predictions over all the predictions. Therefore, it provides a reliable evaluation only for balanced datasets; since this dataset is moderately imbalanced (see Subsection 2.2), it provides general information only;
- **precision** and **recall** introduce parameters that can describe the performance of an estimator on imbalanced datasets. Precision is given by the ratio between the number of true signal events (True Positives, TP) and the total number of events predicted to be signal-like (True Positives + False Positives). On the other hand, recall is defined as the ratio of the number of true signal events (TP) to the actual number of signal events (True Positives + False Negatives);
- the information provided by precision and recall can be visualized through the **confusion matrix**, which has TP and TN on the diagonal and FP and FN on the anti-diagonal. Therefore, it is expected to be mostly diagonal for a good model;
- the information provided by precision and recall is combined in the **f1-score**, an harmonic mean of precision and recall defined as:

$$f1 = 2 \frac{Precision \times Recall}{Precision + Recall}, \quad (1)$$

and therefore returns high values only if both precision and recall are high;

- **ROC-AUC** is the "Area Under the (ROC) Curve", where the ROC Curve plots the true positive ratio (recall) against the false positive ratio (1 - specificity, where specificity is defined as the ratio between TN and TN + FP); therefore, the higher the AUC, the better the performance.

The main difference between f1-score and ROC-AUC is that the former provides a value based on the predictions made by the model, which implies that a given threshold, depending on which events are labeled as background or signal, is fixed. On the other hand, ROC-AUC score computes the performance of the estimator across all possible classification thresholds. Therefore, it provides a global evaluation that is more suitable for the comparison of different models.

6.2 Evaluation Function

The same evaluation function was applied to the three models taking into account that:

- `roc_auc_score()` takes scores as input. They are calculated with the method `predict_proba()` for SGD (available since it was built with logistic loss function) and XGBoost classifier. This method returns an object with two columns: the first one for the probability that the event is background, the second one for the probability that the event is signal. In this case, only the second column was selected. For the DNN, probabilities are the output of the `predict()` method as the output layer was built with the sigmoid activation function. The output needs to be flattened with `Ravel()` method from NumPy library;
- the methods `accuracy_score()`, `precision_score()`, `recall_score()` and `f1_score()` take predictions as input. They are calculated with the `predict()` function for SGD and XGBoost classifiers. The predict function uses 0.5 as threshold by default. On the other hand, for the DNN it is necessary to work out predictions by applying a boolean condition to probability using the previous threshold.

A comment: the `accuracy_score()`, `precision_score()`, `recall_score()`, `f1_score()` and `roc_auc_score()` are taken from the Scikit-Learn library (module `sklearn.metrics`), as well as the `predict()` and `predict_proba()` methods. The `predict()` method applied to the DNN is taken from the `tensorflow.keras` library.

6.3 Results

Results of evaluation were listed in Table 2, while the ROC curve and the confusion matrix were shown in Figure 5 for SGD, in Figure 6 for XGBoost and Figure 7 for the DNN.

6.4 Comments and analysis of the results

From Table 2 it is possible to state that the overall performance of the SGD Classifier is worse compared to the performances of XGBoost and of the DNN. As mentioned in Section 5, the SGD Classifier works with regularized linear models that are unable to capture the non-linear relationships between the features and the labels in the dataset of the Higgs Boson ML competition, which makes it unsuitable for this classification task and justifies the poor

Metric	SGDC	XGBoost	DNN
Accuracy	0.75	0.84	0.84
Precision	0.67	0.79	0.79
Recall	0.55	0.71	0.72
F1-score	0.60	0.75	0.76
ROC-AUC	0.81	0.90	0.91

Table 2: Results of evaluation of the SGD Classifier, the XGBoost classifier and the Deep Neural Network on the test set (scaled for SGDC and DNN). Results were truncated at the second significant digit.

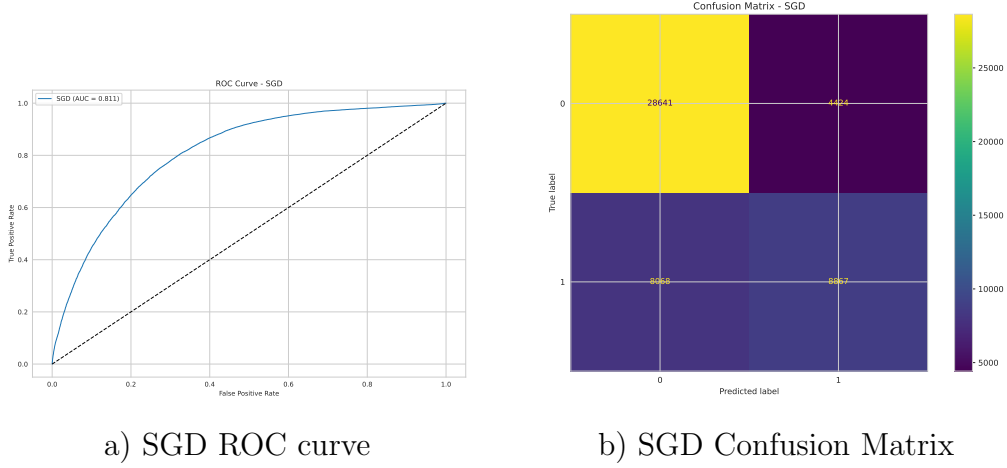


Figure 5: a) ROC Curve (left) and b) Confusion Matrix (right) for SGD Classifier. In the ROC curve, the AUC is 0.810; while in the confusion matrix, the number of true negatives (top left box) is 28176, the number of true positives (bottom right) is 9422, the number of false positives (top right box) is 4691 and the number of false negatives (bottom left) is 7711.

performance. On the other hand, XGBoost and the DNN are capable of implementing non-linear algorithms that can capture these non-linear relations; therefore, the performance significantly improved and it is basically equivalent between the two. However, the training time required by XGBoost is significantly smaller, as shown in Table 3.

In particular, the training time for the DNN is more than seven times larger than that of XGBoost, and this result worsens if the number of epochs is increased to improve the AUC score. These considerations make XGBoost a more suitable candidate to take on the Higgs Boson Machine Learning Challenge.

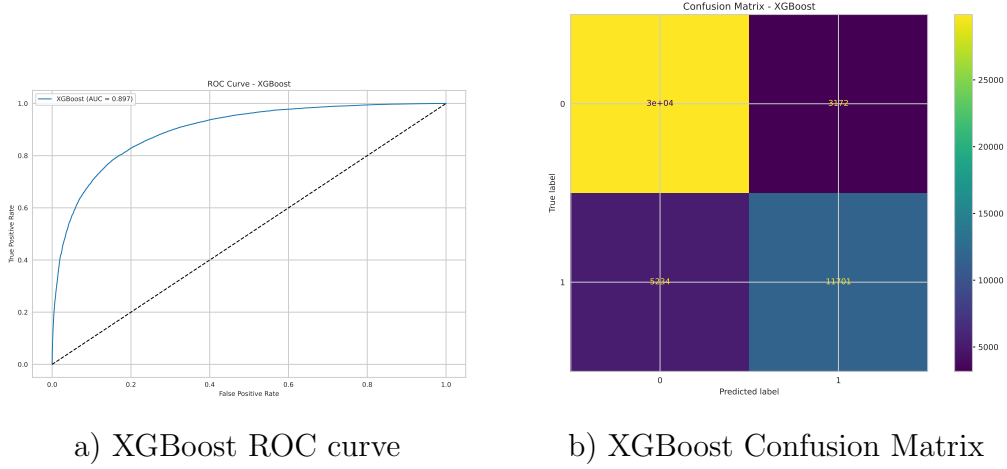


Figure 6: a) ROC Curve (left) and b) Confusion Matrix (right) for XGBoost Classifier. In the ROC curve, the AUC is 0.903; while in the confusion matrix, the number of true negatives (top left box) is 3×10^4 , the number of true positives (bottom right) is 12239, the number of false positives (top right box) is 3269 and the number of false negatives (bottom left) is 4894.

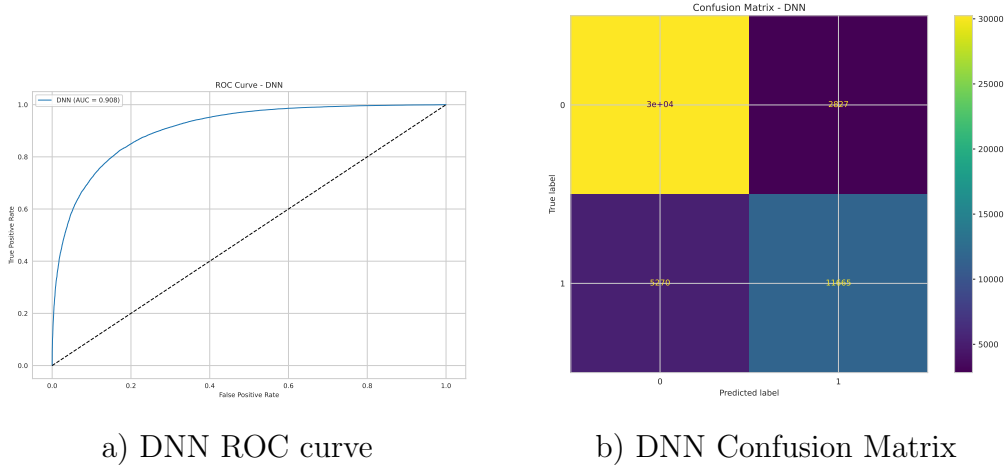


Figure 7: a) ROC Curve (left) and b) Confusion Matrix (right) for DNN. In the ROC curve, the AUC is 0.907; while in the confusion matrix, the number of true negatives (top left box) is 3×10^4 , the number of true positives (bottom right) is 12422, the number of false positives (top right box) is 3321 and the number of false negatives (bottom left) is 4711.

7 Improvements to the XGBoost Classifier

As of now, XGBoost potential has not been fully maximized yet, as a very naive implementation was performed in order to compare its scores with those

Model	ROC-AUC	Training Time (s)
XGBoost	0.903	16.5
DNN	0.907	122

Table 3: Comparison of the performance of the XGBoost and DNN estimators in terms of ROC-AUC score (first column) and training time (second column), measured as wall time (s). Training time values refer to CPU run-time type.

of SGD and DNN. Below, three possible improvements were developed independently. The corresponding XGBoost classifiers were evaluated according to the same criteria that were described in Subsections 6.1 and 6.2.

7.1 Model 1: Dataset with `np.nan` values

As mentioned in Section 4, the XGBoost classifier is capable of handling `np.nan` values well; therefore, the possibility to improve its score was investigated by keeping `np.nan` values in the dataset (`X_train`) without replacing them. In particular, retaining the original missing values instead of imputing them avoids introducing artificial bias or noise into the dataset, allowing the model to treat the absence of information itself as a potentially predictive pattern.

Another classifier, `xgb_model_1`, was defined to be trained on this dataset `X_train` and was referred to as Model 1.

7.2 Model 2: Non-linear feature selection

The function `mutual_info_classif()` of the `scikit-learn` library (in the `sklearn.feature_selection` module) was used to quantify non-linear relationships between each feature and the target. In particular, this is done by calculating mutual information: in this sense, the procedure is intuitive and straightforward, and seemed suitable for an effective feature selection. To add to this, `mutual_info_classif()` seemed more appropriate than any other linear selector as the analysis of the performances in Subsection 6.3 proved that the features in the dataset are mainly non-linearly related to the target labels.

The aim of this section is to investigate how features with low mutual information score affect the performance of the XGBoost classifier. Therefore, the ten features with the highest scores were selected for training using a `SelectKBest()` object with "mutual_info_classif" as score function. To reduce execution time, this object was applied to a subset of 20000 events.

The scores were listed Table 4, from which it is possible to observe that by keeping only the first ten features, the dataset can be reduced significantly while maintaining a wide range of mutual information scores. Therefore, not only could this solution improve the ROC-AUC score by removing features that have poor predictive power, but also the training time of the XGBoost classifier could be reduced.

Feature	Score
DER_mass_MMC	0.14
DER_mass_transverse_met_lep	0.09
DER_mass_vis	0.08
PRI_tau_pt	0.05
DER_met_phi_centrality	0.05
DER_pt_ratio_lep_tau	0.04
DER_mass_jet_jet	0.03
DER_deltaeta_jet_jet	0.03
DER_lep_eta_centrality	0.03
DER_prodelta_jet_jet	0.03

Table 4: Top ten features for mutual information score. Values were truncated at the second significant digit.

Therefore, another classifier, `xgb_model_2`, was defined to be trained on the dataset `X_tr_2` that contained the top ten features only, and was referred to as Model 2.

7.3 Model 3: Hyperparameter tuning

The learning process of the XGBoost classifier is controlled by hyperparameters, which are parameters that are passed as arguments through the constructor. As of now, all the XGBoost classifier that were defined used default values for hyperparameters. However, their values can be optimized with hyperparameter tuning, which was performed with `RandomizedSearchCV` (from `scikit-learn` library, module `sklearn.model_selection`). Differently from `GridSearchCV`, which tries all possible combinations of parameters, `RandomizedSearchCV` only tries a given number (in this case `n_iter = 10`) of parameter combinations, which reduces runtime significantly. Among these, it selects the one that gives the best (ROC-AUC, in this case) score. In particular, the score is obtained via cross validation, with a split of the training set (`X_train`) into five (in this case) folds. As hyperparameter tuning can be extremely computationally intense, the search space was defined focusing

on four key hyperparameters that control overfitting and underfitting, as this criticality had not been handled yet in any of the XGBoost models presented so far.

- **max_depth**: limits the maximum depth of each decision tree, controlling its complexity. A range of integers between 3 and 10 was selected to explore models, starting from simple structures up to moderately complex ones;
- **learning_rate**: shrinks the contribution of each tree. It was sampled from a uniform distribution between 0.01 and 0.1, as lower values generally lead to better generalization;
- **subsample**: represents the fraction of training samples randomly selected to train each tree. Allowing this parameter to vary between 0.5 and 1.0 bagging-like regularization is introduced, making the ensemble more robust against noise;
- **n_estimators**: specifies the total number of sequential trees (boosting rounds). A range between 50 and 200 was defined to provide sufficient model capacity without excessively increasing computational overhead.

In Table 5 the set of best-performing parameters identified by the search was listed.

In particular, results from Table 5 suggest that the best-performing model is

max_depth	learning_rate	subsample	n_estimators
5	0.05	0.87	158

Table 5: Best hyperparameters of a XGBoost Model obtained by tuning with `RandomizedSearchCV` on the Higgs Boson Machine Learning Challenge training set.

kept relatively simple as it only requires five-level-deep trees, which reduces overfitting and makes the model fast and light. Overfitting is also controlled by the low learning rate, which suggests a gradual learning from mistakes. This structure is supported by **n_estimators** = 158, which provides sufficient ensemble capacity. Finally, the high **subsample** (0.87) indicates that while the model utilizes almost the entire dataset for stability, the inclusion of a slight stochastic variation helps in smoothing out potential noise.

Another classifier, **xgb_tuned**, which takes in input the best-performing parameters of Table 5, was defined to be trained on the initial dataset **X_tr** and was referred to as Model 3.

7.4 Evaluation of Model 1, Model 2, Model 3

Model 1, Model 2 and Model 3 were evaluated according to the same approach as explained in Subsections 6.1 and 6.2. However, in this case, the evaluation directly focused on the AUC-SCORE and the execution time, the most relevant parameters to take into account. In particular, Model 1, Model 2 and Model 3 were compared to the first XGBoost model that was built in Section 5.1, which was referred to as Model 0. From Table 6, it is possi-

	ROC-AUC	Training Time (s)
Model 0	0.90	18.3
Model 1	0.89	9.73
Model 2	0.89	11.8
Model 3	0.90	13.9

Table 6: Results of evaluation in terms of ROC-ACU (first column) and training time (second column) for Model 0 (developed in Subsection 5.1), Model 1, Model 2 and Model 3. Training time is expressed in terms of wall time (s).

ble to see that no attempt improved the XGBoost performance in terms of ROC-AUC score. However, on the one hand, both Model 1 and Model 2 improved the XGBoost performance in terms of training time significantly. On the other hand, hyperparameter tuning with `RandomizedSearchCV` was extremely time consuming, as execution lasted for 445 s.

However, the investigation could be led further if a model was trained by taking into account all the improvements that have been suggested contemporaneously, with some additional advice:

- Feature selection must be performed with methods that can handle `np.nan` values in the dataset;
- hyperparameter search can be performed with more modern frameworks such as `Optuna`, which could lead to better performances while requiring less computation time.

8 Project Reproducibility

To ensure that this project could run on any computer without dependency errors, the entire pipeline was containerized.

The project used a Docker container that automatically installs all the necessary Python libraries (such as `Scikit-learn`, `xgboost`, and `tensorflow.keras`).

When the container is started, the training dataset and the main python script must be mounted in the container, which automatically runs the script. This approach ensures that the results are 100% reproducible on any system.

9 Conclusions

This project compared three machine learning approaches—SGD, XGBoost, and a Deep Neural Network—on the Higgs Boson Machine Learning Challenge dataset. While the Deep Neural Network achieved the highest ROC-AUC score, XGBoost proved to be the most effective overall solution thanks to its comparable predictive performance and significantly shorter training time. Additional experiments on the XGBoost model showed that simple improvements, such as handling missing values differently, feature selection, or hyperparameter tuning, did not significantly improve the ROC-AUC score, although they reduced the computational cost in some cases. Overall, this work highlights the importance of evaluating machine learning models not only in terms of predictive performance but also with respect to computational efficiency and reproducibility. Future work could investigate the combined application of the proposed improvements, more advanced hyperparameter optimization frameworks such as Optuna, and more sophisticated neural network architectures to further improve performance.

References

- Adam-Bourdarios, C., Cowan, G., Germain, C., Guyon, I., Kégl, B., & Rousseau, D. (2015, 13 Dec). The Higgs boson machine learning challenge. In G. Cowan, C. Germain, I. Guyon, B. Kégl, & D. Rousseau (Eds.), *Proceedings of the nips 2014 workshop on high-energy physics and machine learning* (pp. 19–55, Vol. 42). PMLR. <https://proceedings.mlr.press/v42/cowa14.html>
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Jia, Y., Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, . . . Xiaoqiang Zheng. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software available from tensorflow.org]. <https://www.tensorflow.org/>
XGBoost Developer. (2026). Xgboost release 3.3.0, 108–122.